

Academic Summarizer & Quiz Generator

[Training Datasets](#) · [Data Preprocessing](#) · [Feature Extraction](#)
[Firebase Storage](#) · [Algorithms](#) · [Pre-trained Models](#)

Focused Technical Reference — Complete Codebase Analysis
Source: uploads.zip

1. Training Datasets

IMPORTANT: This project does NOT train any model from scratch. It uses pre-trained models and statistical algorithms that operate directly on the user's input document at runtime. There is NO custom training dataset stored in, downloaded by, or shipped with this codebase.

The pre-trained models used inside this project were trained on specific well-known public datasets. The table below documents every dataset involved across the full stack.

Component	Pre-trained Model	Training Dataset	Trained By
Sentence Splitting	NLTK Punkt	Large English text corpus (newspaper + books) — learns abbreviation lists	NLTK project (Kiss & Strunk 2006)
Tokenisation & NER	spaCy en_core_web_sm	OntoNotes 5.0 — ~1M words: news, broadcast, web; annotated with POS, parse trees, NER	Explosion AI (spaCy team)
Abstractive Summary (optional)	sshleifer/distilbart-cnn-12-6	CNN/DailyMail — 300K+ news article + human summary pairs	Facebook AI (BART) + Sam Shleifer (distillation)
Question Generation (optional)	valhalla/t5-base-qg-hl	SQuAD v1.1 — 100K+ Wikipedia paragraph + question-answer triples	Google Research (T5) + Valhalla fine-tune
TF-IDF Vectoriser	scikit-learn TfidfVectorizer	NO pre-training — computed fresh on each input document at runtime	No dataset; purely algorithmic
TextRank / PageRank	NetworkX PageRank	NO pre-training — graph built at runtime from current document sentences	No dataset; purely algorithmic
Keyword Extraction (YAKE)	YAKE library	NO pre-training — statistical features computed from input document	No dataset; unsupervised
Keyword Extraction (RAKE)	rake-nltk	NO pre-training — stopwords list + co-occurrence scoring on input doc	NLTK stopwords list only

1.1 CNN / DailyMail Dataset (for distilbart-cnn-12-6)

Size: ~300,000 article-summary pairs. Collected from CNN and Daily Mail news websites by Karl Moritz Hermann et al. (2015). Each record contains a full news article and its bullet-point highlights. BART was fine-tuned on this to learn how to compress long documents into fluent shorter summaries — giving the abstractive refinement step its paraphrasing ability.

1.2 SQuAD v1.1 Dataset (for valhalla/t5-base-qg-hl)

Size: ~100,000 question-answer pairs. Stanford Question Answering Dataset — crowdworkers read Wikipedia paragraphs and wrote questions. The valhalla fine-tune reverses this: given a paragraph and an answer span, generate the question. This is how the transformer flashcard mode works.

1.3 OntoNotes 5.0 (for spaCy en_core_web_sm)

Size: ~1 million words. Sources: Wall Street Journal, broadcast news (CNN, ABC), broadcast conversations, web text, telephone conversations. Annotated with part-of-speech tags, parse trees, named entities, word senses, and coreference chains.

2. Data Preprocessing

There is no offline preprocessing pipeline — preprocessing happens live on each user request. Raw document bytes go through three sequential preprocessing stages before any NLP algorithm runs.

2.1 Stage 1 — File to Raw Text (extractor.py)

Format	Primary Library	Fallback	Strategy
PDF	pdfplumber (pdfminer.six)	PyMuPDF (fitz)	Iterate pages -> page.extract_text(). If result < 50 chars, try PyMuPDF page.get_text('text'). Both fail -> ExtractionError.
DOCX	python-docx	None	Open OOXML ZIP -> iterate doc.paragraphs -> collect .text -> join with newlines. Tables/headers not extracted.
TXT	Built-in open()	None	Try UTF-8 -> Latin-1 -> CP-1252 in order. First success wins. All fail -> ExtractionError.

2.2 Stage 2 — 20-Step Regex Cleaning (cleaner.py)

20 compiled regular-expression transformations run in fixed order. Without this, PDF noise would corrupt every downstream algorithm.

#	Transformation	Why Needed	Pattern
1	Unicode NFKD normalisation	Convert accented chars (e with accent -> e), strip non-ASCII	unicodedata.normalize + ascii encode/decode
2	Hyphenated line-break fix	PDFs split 'algo-[newline]rithm' across lines	-\\n\\s* -> empty
3	Mid-sentence newline -> space	Column layout breaks sentences mid-line	(?<=[a-z])\\n(?=[a-z]) -> space
4	Paragraph break normalise	3+ blank lines -> 2 standard paragraphs	\\n{3,} -> \\n\\n
5	URL removal	URLs are not content sentences	https?:/\\S+ www\\.\\S+
6	Email removal	Addresses are noise	\\S+@\\S+\\.\\S+
7	Section heading strip	'Step 1 -' labels are structural, not content	^(Step Phase Part)\\s+\\d+.*?[-:]
8	Numbered step strip	Leading '1.' or '3)' at line start	^\\d+[\\.\\)]\\s+
9	Citation bracket removal	[1], [2,3] academic citations	\\[\\d+(?:,\\s*\\d+)*\\]
10	Citation parens removal	(2020) year citations	\\(\\d{4}\\)
11	Bullet symbol strip	•, ., arrows, boxes at line starts	^[bullet chars]\\s*
12	Page number removal	'Page 5' or '3/12' patterns	\\bPage\\s+\\d+\\b
13	Standalone number removal	Lines with only 1-4 digits (footer page numbers)	^\\d{1,4}\$

#	Transformation	Why Needed	Pattern
1 4	Ellipsis normalise	3+ dots -> single period	\.{3,} -> .
1 5	Special char removal	Non word/space/punctuation -> space	[^\w\s.,;:!?'"-]
1 6	Math expression removal	Inline algebra biases scoring	\b[a-z]\s*[=+\.*/]\s*[a-z0-9]
1 7	Subscript removal	H2O, x_n style tokens	[a-zA-Z]\d+ [a-zA-Z]_[a-zA-Z]
1 8	Inline formula removal	Short letter fragments from formulas	Short alpha-only sequences
1 9	Trailing space strip	Trailing whitespace per line	\s+\$ per line
2 0	Multi-space collapse	2+ spaces -> 1 space	\s{2,} -> space

2.3 Stage 3 — Validity Gate (cleaner.py: is_text_valid())

Check	Threshold	Rationale
Word count	Minimum 100 words	Fewer than 100 words cannot produce a meaningful 4-sentence summary
Alpha ratio	>=40% of all chars must be alphabetic	Pages of tables, formulas, or code fail this — not suitable for NLP summarisation

2.4 Stage 4 — Sentence Splitting (tokenizer.py)

NLTK's `sent_tokenize()` applies the Punkt model to split clean text into sentences. Three additional filters apply per sentence:

Filter	Rule	Reason
Too short	Word count < 6 -> discard	Likely a heading, label, or fragment
Too long	Word count > 80 -> discard	Likely a merged table row or PDF artifact
Alpha ratio	Alpha chars < 60% -> discard	Mostly symbols, numbers, or formatting noise

2.5 Stage 5 — Lemmatisation (tokenizer.py)

Each surviving sentence is run through `spaCy en_core_web_sm`. Tokens kept only if they pass ALL five conditions simultaneously:

Filter	Condition
Not a stopword	Not in spaCy built-in stopwords AND not in NLTK English stopwords
Not punctuation	<code>Token.is_punct == False</code>
Not whitespace	<code>Token.is_space == False</code>

Filter	Condition
Minimum length	<code>len(token.text) >= 2</code>
Purely alphabetic lemma	<code>token.lemma_.isalpha() == True</code>

Surviving tokens stored as lemma (base form): 'running' -> 'run', 'algorithms' -> 'algorithm'. This reduces vocabulary fragmentation in TF-IDF and frequency scoring.

3. Feature Extraction

Six independent feature signals are extracted for every sentence after preprocessing. These feed directly into the weighted scoring engine.

3.1 Feature 1 — TF-IDF Score (scoring.py)

Library: scikit-learn TfidfVectorizer. All sentences are vectorised (n-gram 1-2, English stop words removed). The row mean of the TF-IDF sparse matrix is each sentence's score. Sentences with rare but document-specific terms score high.

```
vectorizer = TfidfVectorizer(stop_words='english', ngram_range=(1, 2))
matrix = vectorizer.fit_transform(sentences)
scores = matrix.mean(axis=1).A1.tolist() # row means -> Feature 1
```

3.2 Feature 2 — Word Frequency Score (scoring.py)

Algorithm: Normalised term frequency using Python Counter. All preprocessed tokens are pooled. Max frequency normalises to [0,1]. For each sentence: $\text{sum}(\text{norm_freq}[t] \text{ for } t \text{ in tokens}) / \text{len}(\text{tokens})$.

3.3 Feature 3 — TextRank / PageRank Score (textrank.py)

Algorithm: TextRank (Mihalcea & Tarau 2004). Applies Google PageRank to sentences instead of web pages. Steps:

- Build TF-IDF matrix (same vectoriser as Feature 1).
- Compute $n \times n$ cosine similarity matrix via `sklearn.metrics.pairwise.cosine_similarity`.
- Zero diagonal (no self-loops). Zero edges below `SIMILARITY_THRESHOLD = 0.10`.
- Build weighted undirected graph with NetworkX: nodes = sentences, edge = cosine similarity.
- Apply `networkx.pagerank(alpha=0.85, max_iter=200)`. Most 'central' sentences rank highest.
- Normalise to [0,1]. Fallback 1: if TF-IDF fails -> Jaccard similarity. Fallback 2: if PageRank fails to converge -> `degree_centrality()`.

3.4 Feature 4 — Position Score (scoring.py)

Heuristic. No library. Position ratio = $i/(n-1)$ mapped to fixed scores:

Position	Ratio Range	Score	Rationale
Introduction	0.00 - 0.10	1.00	Thesis and key claims stated at start
Early body	0.10 - 0.25	0.80	Background and methodology
Middle body	0.25 - 0.70	0.50	Supporting detail, lower priority
Late body	0.70 - 0.85	0.75	Results and discussion
Conclusion	0.85 - 1.00	1.00	Key findings restated

3.5 Feature 5 — Keyword Score (scoring.py)

Keyword tokens set is built from YAKE+RAKE results (words > 3 chars). For each sentence: overlap count between sentence tokens and keyword set, normalised across all sentences.

3.6 Feature 6 — Entity Density Score (entities.py)

spaCy NER run over full document collects all_ent_set. Per sentence: intersection count of sentence entities with all_ent_set, normalised by maximum count across all sentences.

3.7 Length Multiplier (scoring.py)

Applied as post-multiplication factor to the weighted sum:

Word Count	Multiplier	Meaning
< 8 words	0.70	Fragment — likely incomplete
8 - 12 words	0.70 (clamped minimum)	Short but possibly valid
12 - 45 words	1.00	Ideal academic sentence length
45 - 60 words	0.80	Somewhat long
> 60 words	0.70 (clamped minimum)	Very long — possibly table row or run-on

3.8 Final Score Formula

```
score = (0.30 x tfidf) + (0.20 x frequency) + (0.20 x textrank)
+ (0.10 x position) + (0.10 x keyword) + (0.10 x entity)
score *= length_multiplier
```

3.9 YAKE Keyword Extraction Sub-Features

YAKE Feature	What It Measures
Casing	Capitalised/title-case terms score higher — likely proper nouns or technical terms
Position	Terms in first third of document score higher — authors introduce key concepts early
Frequency	Very frequent terms PENALISED (likely stopwords like 'the', 'is')
Relatedness	How often the term co-occurs with other terms in a sliding context window
DifSentence	How spread across different sentences — widespread = more important

4. Firebase & Data Storage

FINDING: There is NO Firebase integration in this codebase. Searching `app.py`, all `services/*.py`, `requirements.txt`, `.env`, and `.env.example` found ZERO references to Firebase, Firestore, firebase-admin, or any cloud database.

4.1 What the Code Actually Does for Storage

A simple local temporary file pattern is used. No persistent database of any kind is involved:

Action	Location	Duration	Code Location
User uploads PDF/DOCX/TXT	Local disk: uploads/ folder	Only during extraction — deleted immediately after text is extracted	<code>app.py -> _save_and_extract()</code>
Extracted text	Python memory (RAM)	Duration of request only — not written to disk	In-memory string variable
Computed scores/features	Python memory (RAM)	Duration of request only	All <code>services/*.py</code> return values
API response (summary/quiz/flashcards)	HTTP response body -> client	NOT stored server-side at all	<code>Flask jsonify()</code>

Uploaded files are temporarily saved to `uploads/`, text is extracted, then the file is immediately deleted with `os.remove(tmp_path)`. All computed results live only in Python objects for the life of the HTTP request and are returned as JSON.

5. Algorithms Used

5.1 Summarizer Algorithms

Algorithm 1 — TF-IDF

Type: Statistical, unsupervised. Library: scikit-learn. $TF(t,s) = \text{count of term } t \text{ in sentence } s / \text{total terms in } s$. $IDF(t) = \log(N/df(t))$. $TF\text{-}IDF = TF \times IDF$. Row mean = sentence score.

Algorithm 2 — TextRank (PageRank on text graph)

Type: Graph-based, unsupervised. Libraries: NetworkX, scikit-learn, NumPy. Sentence similarity via cosine of TF-IDF vectors. Damping factor $\alpha=0.85$. Iterates until convergence ($\text{tol}=1e-5$, max 200 iterations).

Algorithm 3 — Cosine Similarity Deduplication

Type: Vector geometry. Library: scikit-learn. Greedy: accept top sentence; for each next candidate compute cosine similarity against accepted set. If any similarity $\geq 0.72 \rightarrow$ discard as near-duplicate.

Algorithm 4 — Position Heuristic

Type: Rule-based. No library. Introduction (0-10%) and conclusion (85-100%) get weight 1.0; middle body (25-70%) gets 0.50.

Algorithm 5 — Keyword Overlap Scoring

Type: Set intersection. No library. Keyword token set intersected with sentence token set. $\text{overlap_count} / \text{keyword_set_size} \rightarrow$ normalised keyword density score.

Algorithm 6 — Named Entity Density Scoring

Type: NER-based counting. Library: spaCy. Count of document-level entities in each sentence, normalised to [0,1].

Algorithm 7 — Weighted Linear Combination

Type: Linear algebra. Pure Python. $\text{score} = (0.30 \times \text{tfidf}) + (0.20 \times \text{freq}) + (0.20 \times \text{textrank}) + (0.10 \times \text{position}) + (0.10 \times \text{keyword}) + (0.10 \times \text{entity})$. Then multiplied by length_multiplier .

5.2 Flashcard Generator Algorithms

Algorithm 8 — Definition Pattern Matching

Type: Regular expression. Library: Python re. Nine compiled patterns match sentences like 'X is a Y', 'X refers to Y', 'X consists of Y'. $\text{group}(1) = \text{question subject}$, $\text{group}(2) = \text{definition answer}$. Generic or overly long subjects filtered.

Algorithm 9 — Cause-Effect Pattern Matching

Type: Regular expression. Library: Python re. Three patterns match causal language: 'X because Y', 'X leads to Y', 'X improves/reduces Y'. Groups become question and answer.

Algorithm 10 — Comparison Word Detection

Type: Set intersection. No library. Signal words: {unlike, whereas, while, however, compared, contrast, difference, similarly, both, either, neither}. Any match $\rightarrow$ comparison flashcard.

Algorithm 11 — T5 Question Generation (Optional Transformer)

Type: Seq2seq generation. Library: HuggingFace Transformers. Input format: 'answer: {answer_span} context: {sentence}'. Answer span found by spaCy NER (preferred) or first noun chunk. Beam search: $\text{num_beams}=4$,

early_stopping=True, max_length=64.

5.3 Quiz Generator Algorithm

Algorithm 12 — MCQ Construction with Random Distractors

Type: Rule-based + random sampling. Library: Python random.

- Step 1: Run definition + keyword generators to get question-answer base pairs.
- Step 2: Correct answer = the card's answer string.
- Step 3: All other sentences (≥ 6 words, not the correct answer) = distractor pool.
- Step 4: `random.shuffle()` the pool, take first 3.
- Step 5: Shuffle [correct + 3 distractors] into A-D order with `random.shuffle()`.
- Step 6: `correct_letter = 'ABCD'[options_list.index(correct_answer)]`.

6. Pre-trained Models Used

Model	Status	Size	Source
spaCy en_core_web_sm	ALWAYS ACTIVE (required)	~12 MB	Explosion AI
NLTK Punkt Tokeniser	ALWAYS ACTIVE (required)	~1 MB	NLTK project
sshleifer/distilbart-cnn-12-6	OPTIONAL (abstractive mode)	~1.2 GB	HuggingFace Hub
valhalla/t5-base-qg-hl	OPTIONAL (transformer flashcards)	~900 MB	HuggingFace Hub

6.1 spaCy en_core_web_sm (Always Active)

Property	Detail
Full name	en_core_web_sm — English, small, OntoNotes 5.0 + web text
Architecture	CNN token embeddings + transition-based dependency parser + CRF NER decoder
Parameters	~12 million
Training data	OntoNotes 5.0 (~1M annotated English words)
Tasks	Tokenisation, POS tagging, Dependency parsing, Lemmatisation, NER
NER labels used	PERSON, ORG, GPE, DATE, LOC, NORP, WORK_OF_ART, EVENT
How loaded	spacy.load('en_core_web_sm') — lazy-loaded and cached as singleton
Used in	tokenizer.py (lemmatisation), entities.py (NER scoring), flashcard_generator.py

6.2 NLTK Punkt Tokeniser (Always Active)

Property	Detail
Algorithm	Unsupervised Punkt (Kiss & Strunk 2006) — learns abbreviation lists from period-token statistics
What it learns	Which period-followed tokens are abbreviations ('Dr.', 'et al.', 'Fig.') vs sentence boundaries
Training data	Large English text corpus (newspapers + books)
How loaded	nlTK.download('punkt') on first run; then sent_tokenize(text)
Used in	tokenizer.py -> split_sentences(); flashcard_generator.py -> sent_tokenize()

6.3 sshleifer/distilbart-cnn-12-6 (Optional)

Property	Detail
Base architecture	BART — Bidirectional Autoregressive Transformer. Encoder bidirectional (like BERT); Decoder left-to-right (like GPT)
Distillation	Distilled from facebook/bart-large-cnn. 12 encoder + 6 decoder layers. ~60% of large BART's parameters

Property	Detail
Parameters	~306 million
Fine-tune dataset	CNN/DailyMail — 300K+ news article to highlight-summary pairs
Inference params	max_length=150, min_length=50, do_sample=False (greedy beam decoding)
Download size	~1.2 GB (cached in ~/.cache/huggingface/ after first download)
Trigger	API request includes use_abstractive_refinement: true
Fallback	If transformers/torch not installed or inference fails -> returns original extractive summary; used_abstractive_refinement: false
Called in	services/abstractive.py -> refine_abstractively()

6.4 valhalla/t5-base-qg-hl (Optional)

Property	Detail
Base architecture	T5-base (Raffel et al. 2019) — Text-To-Text Transfer Transformer. Every NLP task framed as text->text
Parameters	~220 million (T5-base)
Fine-tune dataset	SQuAD v1.1 — 100K+ Wikipedia paragraph + Q&A; triples (task reversed: given answer+context, generate question)
Input format	'answer: {answer_span} context: {full_sentence}'
Output format	A natural language question string targeting the answer span
Inference params	max_length=64, num_beams=4, early_stopping=True
Answer span selection	Prefer spaCy PERSON/ORG/GPE/PRODUCT entity; fallback to first noun chunk
Download size	~900 MB (cached in ~/.cache/huggingface/ after first download)
Trigger	Flashcards API request includes use_transformer: true
Fallback	If model unavailable -> falls back to rule-based flashcard generation automatically
Called in	services/flashcard_generator.py -> _generate_with_transformer()

7. Quick Reference — All Questions Answered

Question	Answer
Does this project train any model?	NO — all models are pre-trained by third parties and loaded as-is.
What datasets were used for training?	OntoNotes 5.0 (spaCy NER), CNN/DailyMail (distilbart), SQuAD v1.1 (T5/valhalla), Punkt corpus (NLTK). All for pre-trained models only — not in the codebase.
Is there a custom training dataset in the zip?	NO — zero training data files exist in the codebase.
How is data preprocessed?	20-step regex cleaning -> validity gate (100 words + 40% alpha) -> NLTK sentence split -> spaCy lemmatisation. All happens live, per request.
What features are extracted?	TF-IDF score, word frequency, TextRank/PageRank, position heuristic, keyword overlap, entity density — plus a length multiplier.
Is Firebase used?	NO — not a single Firebase import, reference, or credential anywhere in the codebase.
How is data stored?	Files saved temporarily to uploads/ disk, deleted immediately after text extraction. All computed results in RAM for request duration only. Nothing persisted.
What algorithm does summarization?	Weighted combination: TF-IDF(0.30) + frequency(0.20) + TextRank(0.20) + position(0.10) + keyword(0.10) + entity(0.10), multiplied by length factor. Then cosine-similarity deduplication.
What algorithms generate flashcards?	Rule-based: regex definition/entity/keyword/cause-effect/comparison patterns. Optional: T5-base-qg-hl transformer with beam search.
What algorithm generates quizzes?	Definition pattern matching for correct answers + random sentence sampling for 3 distractors, shuffled into A-D MCQ format.
Required pre-trained models (always)	spaCy en_core_web_sm (~12MB) + NLTK Punkt (~1MB).
Optional pre-trained models	sshleifer/distilbart-cnn-12-6 (~1.2GB) for abstractive summary; valhalla/t5-base-qg-hl (~900MB) for transformer flashcards.